

Chapter 11A: Calculator GUI

Okay gang, we have finally made it to our real-world examples. This chapter is broken into two parts. In part 11A we'll create the GUI (graphic user interface) for a simple 4-function calculator. In part 11B we'll write the code behind that GUI.

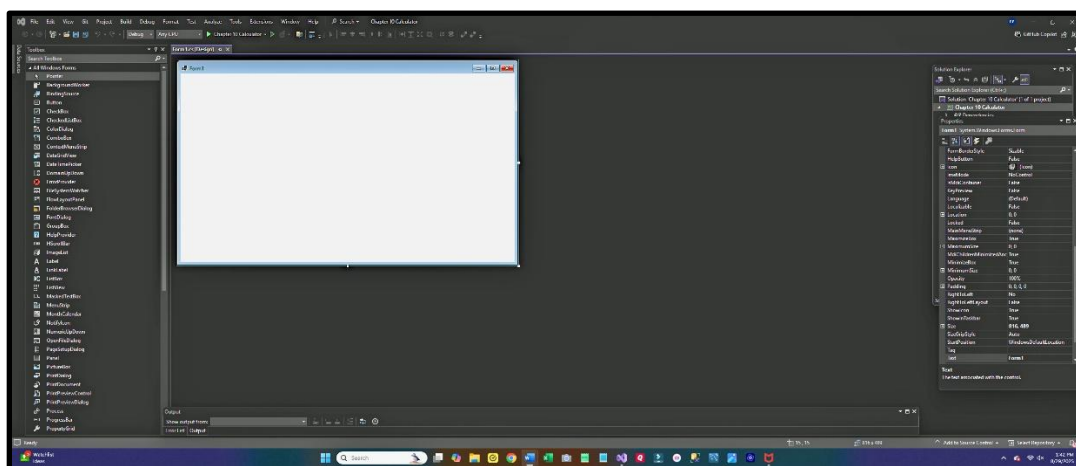
The goal of this example is of course to introduce you all to the graphic elements you have at your disposal when creating web forms and then to further introduce you to event-driven code.

Let's jump to the inside and learn some GUI stuff.

Please begin by creating a new Windows Forms project entitled Chapter 11 Calculator. We'll use this same project for both parts A and B of this example. A screen similar to the one shown here (Figure 11A.1) should be displayed. This Integrated Development Environment, or IDE, is typically characterized by 5 distinct panes. The large area in the middle of the screen with the white form in it is your graphic design area. This window is what will ultimately be displayed to the user. On the right side of the screen are two additional panes. The one on the top is the solution explorer which lists all the files making up your current solution. The bottom-right hand pane displays the properties and methods associated with the currently selected graphic object, in our case the form itself. The pane on the far left is known as your toolbox. You can double click any of the objects listed in the toolbox to add them to your window. Alternatively, you can drag and drop graphic objects onto your form. Finally, the pane at the bottom of the screen is your output pane. This is where Visual Studio sends you messages about how your application is operating.

If you don't have all five panes, you can go up to the View menu option and add them to your screen. Trust me, you are going to want them all.

Figure 11A.1: Visual Studio Standard IDE



Okay, let's start building our calculator application and learning about the various graphic objects available for your use.

Please select your form by single clicking on the window. If you accidentally double click the window you will jump to the code-behind tab. No problem just leave the code there unchanged and click back onto the tab labeled Design.

Once selected we are going to go over to the Properties and Methods pane to set some properties of our to-be-Calculator window. First, go ahead and change the Text property of our Form to Calculator. Next, please change the Size property of your window to 500,600. These values represent the form's width and height in pixels.

Form1		
	Text	Calculator
	Size	500,600

Next, please go over to the Toolbox pane and double click on the PictureBox option. This will add a PictureBox object to your form. As the class name implies, a PictureBox object is used to display an image.

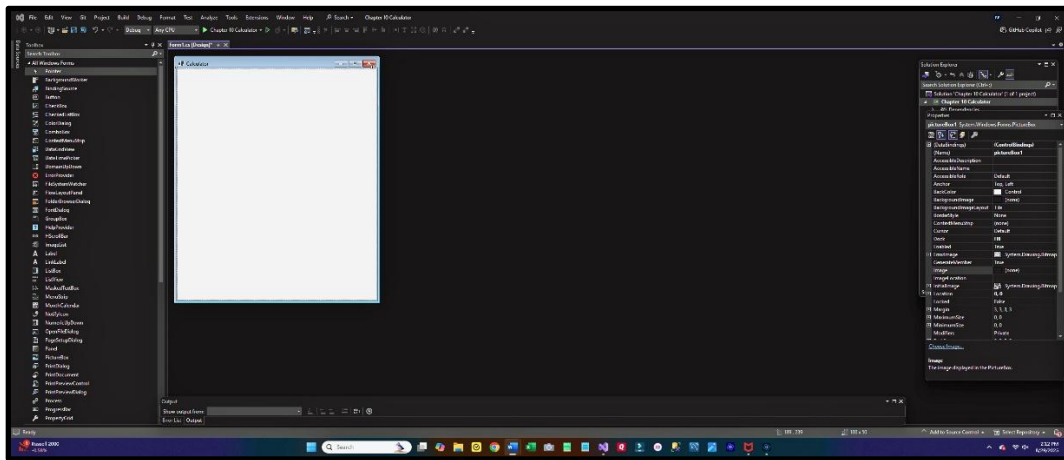
Please drag your new PictureBox object to the center of your form so we can begin working with it. Next, please note the little box with an arrow icon on it in the upper right-hand corner of your PictureBox. Some objects have these very helpful quick property menus. Go ahead and click that box to open the wizard.

From within that little wizard please choose to dock this PictureBox object to its parent object, i.e. the Form. Docking refers to attaching an object to one or more sides of its parent object. In our case, this will allow the user to resize the calculator while simultaneously resizing the calculator's skin. Please note that as an alternative to selecting that option from the wizard you could have gone to the Dock property in the Properties/Methods pane and made your selection there.

Your PictureBox object should now fill your entire form as shown here. (Figure 11A.2)

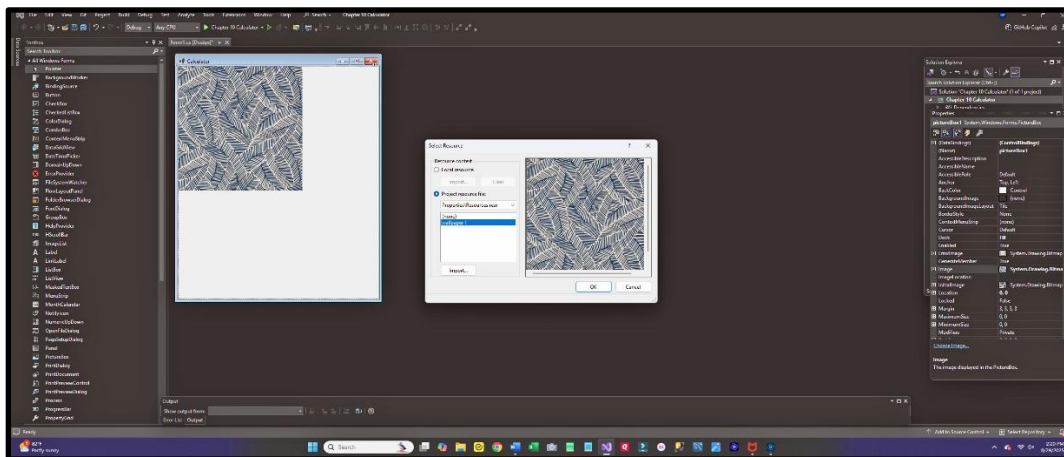
Picturebox1		
	Dock	All Sides
	Image	Your Choice
	SizeMode	StretchImage

Figure 11A.2: PictureBox object Filling Parent Form



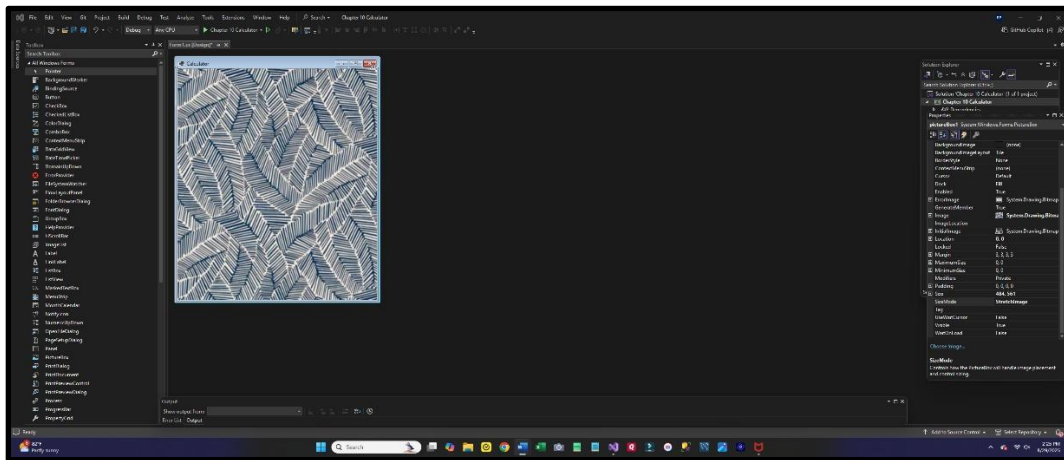
Next, please select the choose image option from the PictureBox's quick menu. A standard file selection window will appear from which you can import any image appropriate to a calculator's skin. I found this cool fern-like image on Google, so I am using it. You can use any image you feel is appropriate. (Figure 11A.3)

Figure 11A.3: Image Selection



Complete setting the properties of your PictureBox calculator skin by setting the SizeMode property to the StretchImage value. As the name implies, this value stretches the image to fit the entire PictureBox area. Because the PictureBox object is docked to all sides of the Form object, the image will now resize with changes to the form's size. (Figure 11A.4)

Figure 11A.4: SizeMode = StretchImage



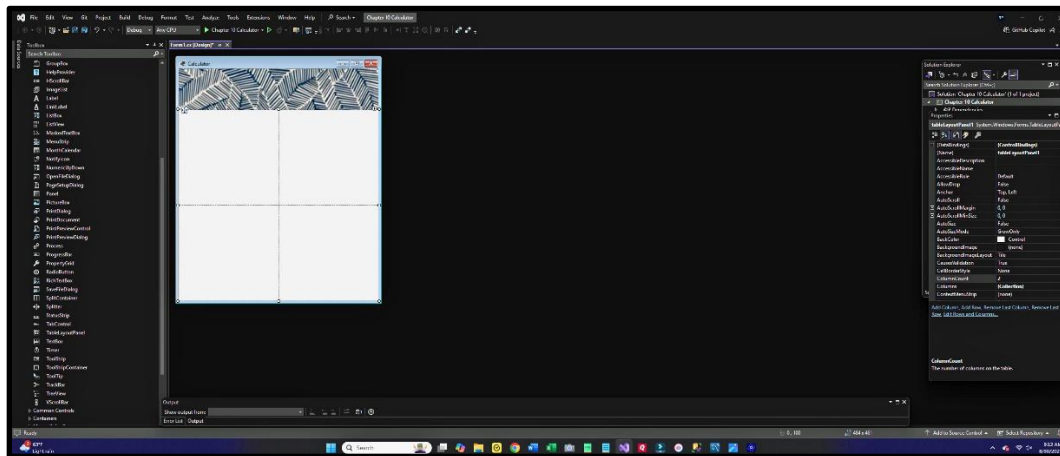
Okay, time to run a test. Click that green arrow at the top of the screen to start executing your new window. Make sure the image you have chosen resizes properly as you resize the window. Cool, mine appears to be working properly.

By the way, I always do a save after each successful test. That way if things go south, I can easily revert back to the last place I know things were working properly.

We are next going to add a container to our form. As the name implies, containers serve as a storage vehicle for other objects. Containers may, or may not, be visible to the user. Our's will not be visible. Okay, please go ahead and add a `TableLayoutPanel` container to your form. We will use this object to align the buttons and number display of our calculator.

Please resize your `TableLayoutPanel` to cover about 80% of the lower portion of the form as shown in this figure. (Figure 11A.5) Please note that to move this object you must click and drag on the little 4-arrow navigation icon in the upper left-hand corner of the object.

Figure 11A.5: TableLayoutPanel



TableLayoutPanel1		
	Size	80% of lower portion of form
	ColumnCount	4
	RowCount	6
	Edit Rows & Columns	
	Column Width	25%
	Row Height	Top 25% Remainder 15%

Using the Properties/Methods pane, change the ColumnCount property of the panel to 4 and the RowCount property to 6. Changing these values creates a grid of 24 cells in your TableLayoutPanel.

Next, we will use the panel's quick properties menu to set the height and width of each of these 24 cells. The menu option you want to use is the "Edit Rows and Columns" submenu. Remember, to open the quick properties menu of any object click the little right arrow in the box in the upper right-hand corner of the object.

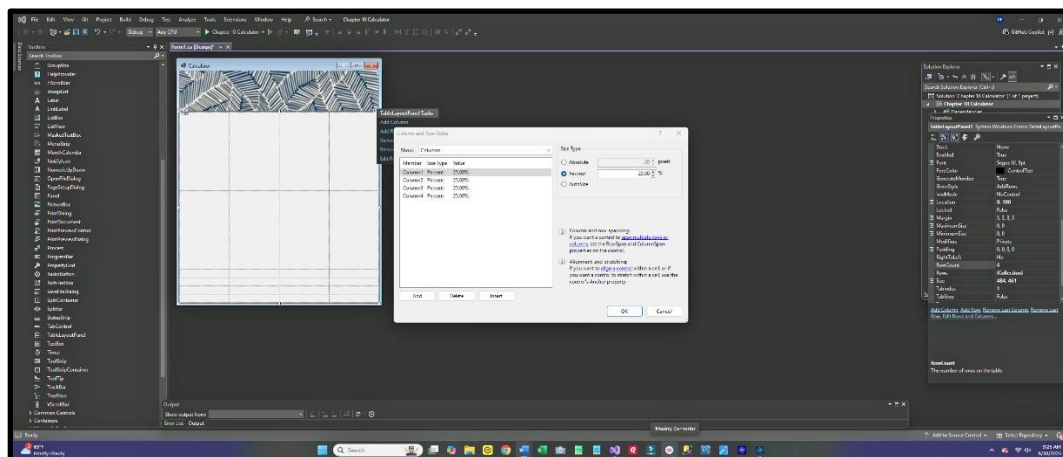


So how wide and tall do we want to make each of the cells of our TableLayoutPanel? Well, when I hit the calculator button on my keyboard, I get a window which looks like this. Let's try to emulate that guy but without the top row of function keys. (We could of course have made those buttons work too, but this chapter is already getting too long.)

It looks like we need a little larger number display at the top spanning all 4 columns and then 5 less tall, but equal height, rows across the bottom.

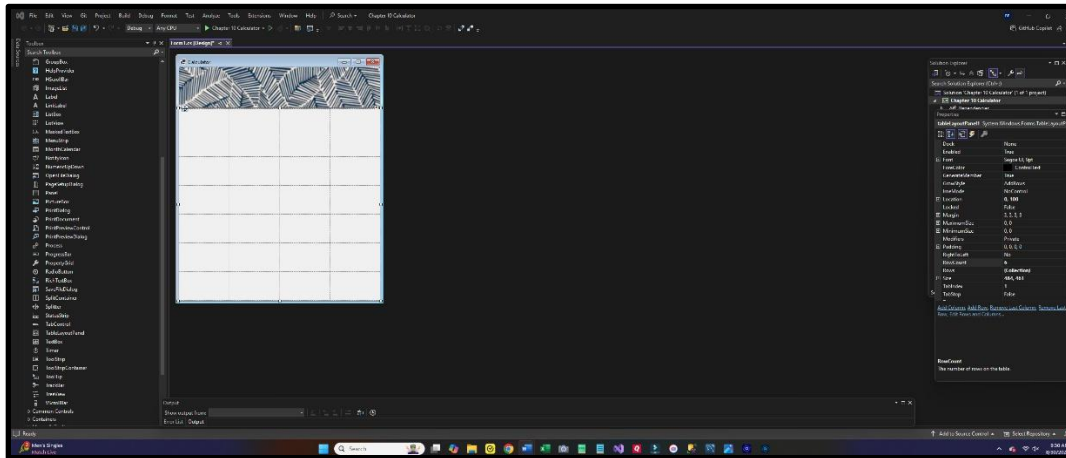
I am starting with the columns, so I just set each of my 4 columns to an equal width of 25%. (Figure 11A.6)

Figure 11A.6: Column Width



Next, using the same wizard please set the row heights. Set the top row to 25 percent and each of the 5 remaining rows to 15% each. (Figure 11A.7)

Figure 11A.7: Row Height



By the way, as an interesting exercise in software testing, go ahead and set the sum of these percentiles to something other than 100%. No beep, no warning message, nothing to inform you that you have made an error. Worse yet, it changes the other values to accommodate for your error. Come on Microsoft, you are better than that!

Enough ranting about Microsoft's questionable software QA. Back to our calculator application. We will begin by adding a Label to the top row of our TableLayoutPanel. First select the TableLayoutPanel with a single click. This will tell the IDE that newly added objects are to be added to the panel object's first open cell.

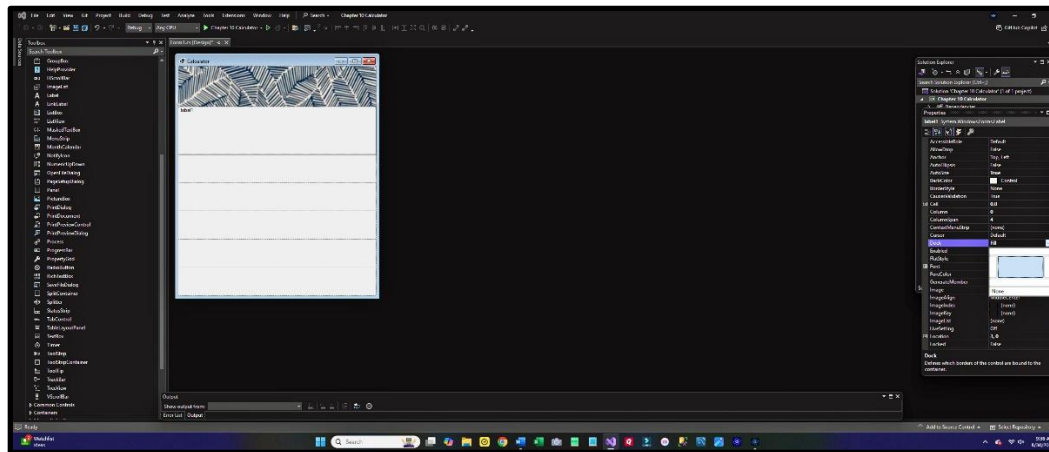
Open the Toolbox and double click on a Label object. Normally, Labels are used to display text. In our application that text will be a series of digits; i.e. a number representing either an operand or a result.

If all went according to plan, a new Label should have been added to the top, left-hand cell of your TableLayoutPanel container. But we want that guy to span all 4 columns of that top row. Well how about we select it and then go the Properties pane and change the ColumnSpan property to 4?

Label1		
	ColumnSpan	4
	Dock	Fill
	Font	Bold, 24pt
	Name	DisplayLabel
	Text	0
	TextAlign	MiddleRight

Please go ahead and change that ColumnSpan property of our Label1 object to 4. What happened? Absolutely nothing. Another Microsoft “undocumented feature” you ask? No, we still need to set its Dock property to Fill to have that guy attached to all sides of the top row. Please go ahead and make that change too. (Figure 11A.8)

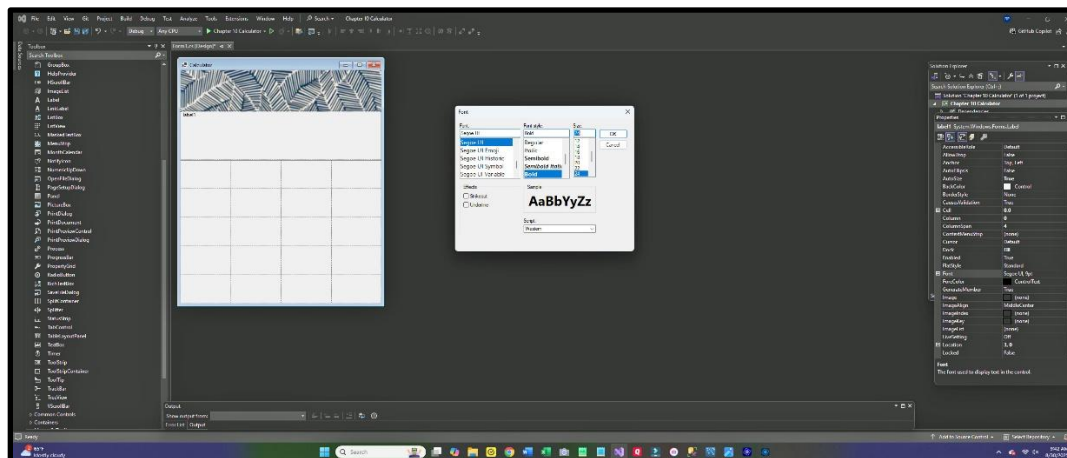
Figure 11A.8: Label Display



By the way, you can either select the appropriate part of those dropdown images to pick your Dock Fill option or you can type the value into the space provided. If you type the value, be sure to capitalize the initial letter. Yes, they are case sensitive because that is really an enum that you are selecting.

We continue working on our Label1 object by updating the font of the text it displays. Begin by ensuring that Label1 is selected and then navigate to the Font property. Click on that property to select it. On the far right side of that property option is a box icon with several small dots across the bottom. Click on that icon to open the Font property wizard. Set our Label1 object Font property values to Bold and 24 point. (Figure 11A.9)

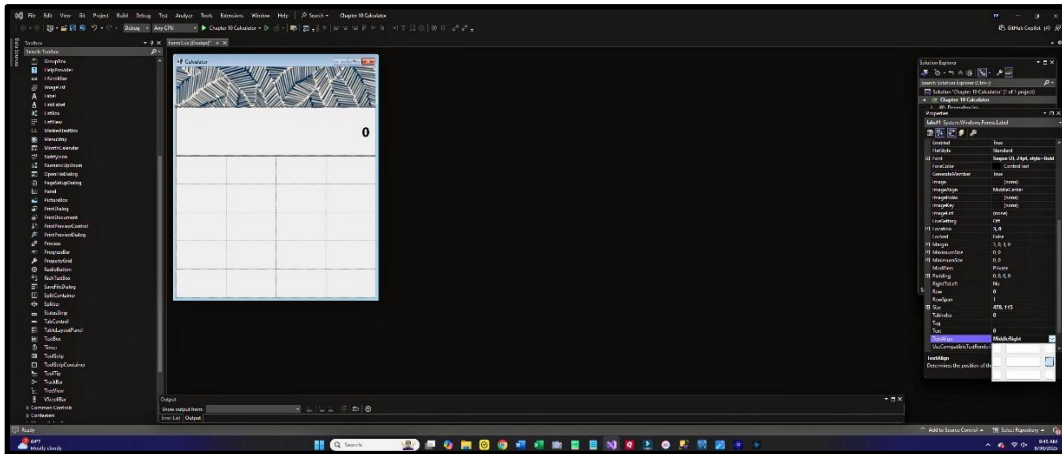
Figure 11A.9: Font Wizard



Next, please change the Name of this label to DisplayLabel. Note, the Name property is always at the top of the properties list regardless of which display method you are using.

Finish your work on this label by changing its Text property to 0 and its TextAlign property to MiddleRight. (Figure 11A.10)

Figure 11A.10: Text and TextAlign



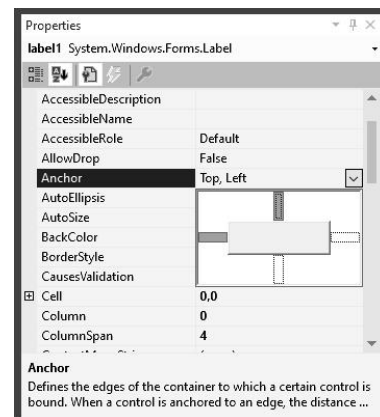
Time for a test. Initial GUI looks fine, but now when I resize the form the TableLayoutPanel object is not resizing with its form parent. What is wrong? We have it's Dock property set to a value of Fill.

So why did the Dock.Fill property not do the trick? Because Dock.Fill works for resizing in all directions only if the child object fills the entire area of the parent object. If it does not, you have to fall back onto the child object's Anchor property.

Select the TableLayoutPanel object and navigate to its Anchor property. Open the dropdown box to see its options and you will see a dropdown graphic similar to the one shown here.

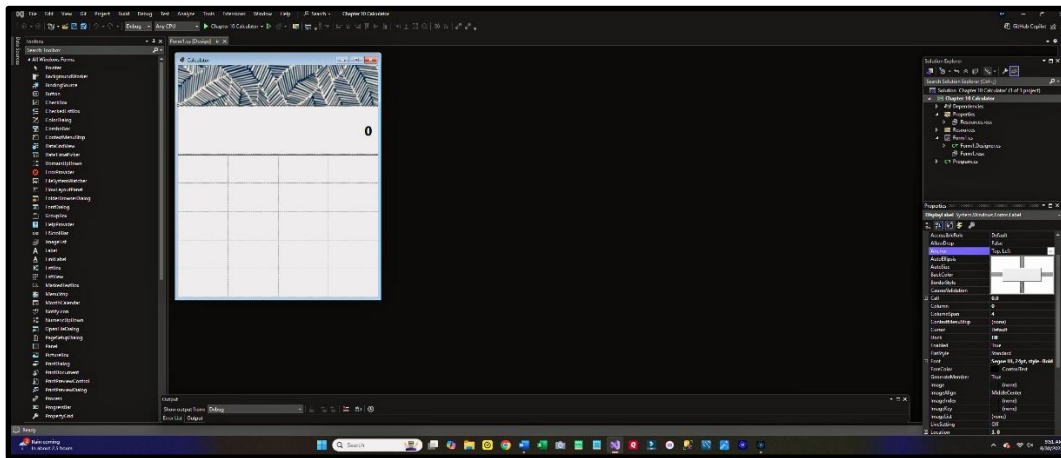
So, what is going on with this graphic? The graphic is supposed to represent a child object, the inner rectangle, placed within, but not totally taking up, the area of the outer parent object. In the image to the right the two solid lines indicate that the child is currently anchored to the top and left sides of the parent.

Under the conditions shown to the right, the child object moves in synch with size changes to the left and top sides of the parent.



We of course want our TableLayoutPanel container to resize in all 4 directions with the parent form so please choose all 4 anchor legs. Try testing and resizing again, everything should now work properly. (Figure 11A.11)

Figure 11A.11: Anchor Property



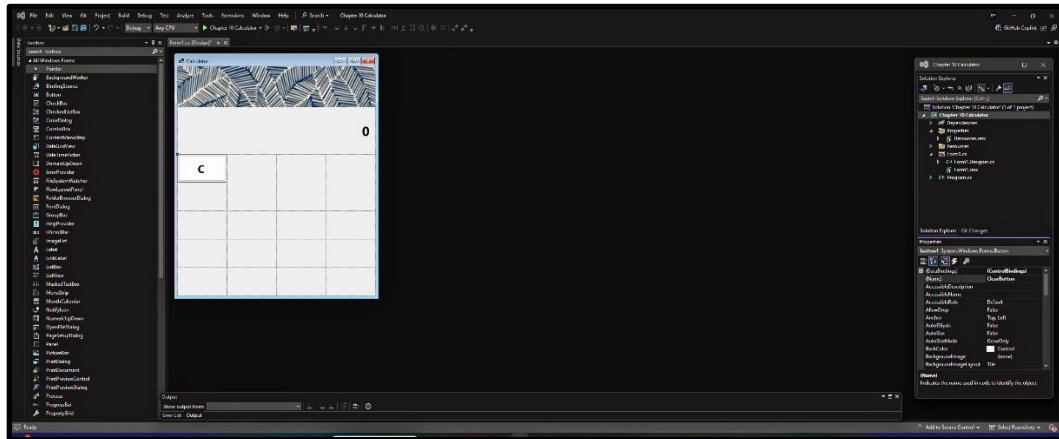
We are next going to add a series of Buttons to the remaining cells of our TableLayoutPanel container. Each Button object will correspond to a button on our calculator.

Next, open the Toolbox and double click on the Button object. Your new Button object should have been placed in the first open cell of the container as shown here. (Figure 11A.12) Let's change the properties of Button1 such that it looks like the clear button on a calculator.

Please make each of the following property changes:

Button1		
	Text	C
	Font	Bold, 20pt
	Dock	Fill
	TextAlign	MiddleCenter
	Name	ClearButton

Figure 11.12: ClearButton



To create the remainder of the buttons for our calculator we are simply going to copy and paste our ClearButton. To copy that guy simply select it with a single click and then simultaneously hit the Ctrl and C keys. Ctrl + C is the standard Windows hot key combination for copy. To paste the new button into the next available cell of our TableLayoutPanel container simply simultaneously hit the Ctrl and V keys. Ctrl + V is the standard Windows hot key combination for paste.

Update the clear entry button properties according to the following:

Button1		
	Name	ClearEntryButton
	Text	CE

Did you notice that we didn't have to change the Font or Dock properties of the ClearEntryButton? The values of those properties were copied over from the ClearButton. Nice time saver.

Repeat the copy/paste process to create the third button in this row. Name the button PercentButton and set its Text property to %.

Button1		
	Name	PercentButton
	Text	%

We will use the final button in this row as our divide key. Name the button DivideButton and make its Text property equal to /.

Button1		
	Name	DivideButton
	Text	/

Go ahead and create the next row of buttons. The button names for that row are NineButton, EightButton, SevenButton and MultiplyButton. The corresponding Text property values are: 9, 8, 7, and *.

Button1		
	Name	NineButton
	Text	9

Button1		
	Name	EightButton
	Text	8

Button1		
	Name	SevenButton
	Text	7

Button1		
	Name	MultiplyButton
	Text	*

The names for the next row of buttons are SixButton, FiveButton, FourButton and AddButton. The corresponding Text property values are of course 6, 5, 4 and +.

Button1		
	Name	SixButton
	Text	6

Button1		
	Name	FiveButton
	Text	5

Button1		
	Name	FourButton
	Text	4

Button1		
	Name	AddButton
	Text	+

Continue by adding another row of buttons to the TableLayoutPanel. The Names for these buttons should be ThreeButton, TwoButton, OneButton and SubtractButton. The Text property values for these buttons should be 3, 2, 1 and -.

Button1		
	Name	ThreeButton
	Text	3

Button1		
	Name	TwoButton
	Text	2

Button1		
	Name	OneButton
	Text	1

Button1		
	Name	SubtractButton
	Text	-

The names for the bottom row of buttons should be ChangeSignButton, ZeroButton, DecimalButton and EqualsButton. The corresponding Text property values are +/-, 0, . and =.

Button1		
	Name	ChangeSignButton
	Text	+/-

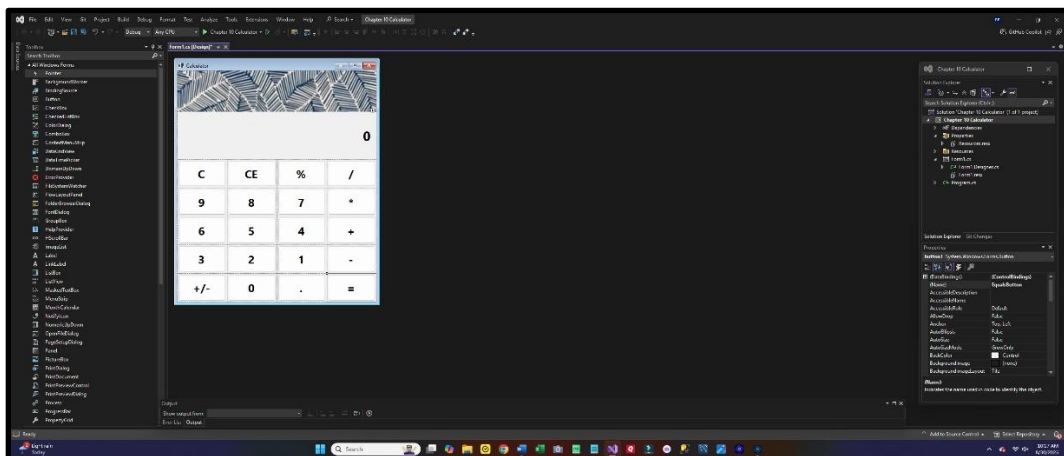
Button1		
	Name	ZeroButton
	Text	0

Button1		
	Name	DecimalButton
	Text	.

Button1		
	Name	EqualsButton
	Text	=

Your GUI, graphic user interface, should now look like this. (Figure 11A.13)

Figure 11A.13: GUI



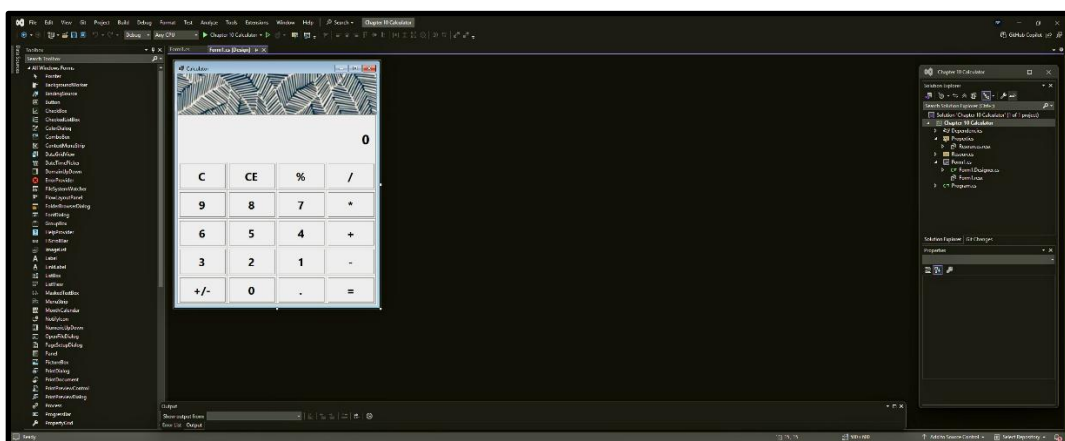
Now when I look at this GUI it looks a little flat to me, so before we conclude let's give this form a little depth. To do so let's first single click on the ClearButton. Then to select all the remaining buttons just depress the CTRL key and single click on each remaining button. When multiple

objects on a form are concurrently selected only those properties which are shared by all those objects are displayed. The property we are interested in is named FlatStyle. We want to set that property's value to PopUp. While we are at it, let's also change the TableLayoutPanel's CellBorderStyle property to InsetDouble. That will also add some depth to our form. By the way, it can be very difficult to click on those container objects so an easier way to select it is via the dropdown menu at the top of the Properties pane. (Figure 11A.14)

All Buttons		
	FlatStyle	PopUp

TableLayoutPanel		
	CellBorderStyle	InsetDouble

Figure 11A.14: Depth to Form



Okay, I think this is a good point at which to stop for this lesson. In the next chapter we will write the code behind this calculator. Let's jump to the outside and do a quick review of what we learned.

Well, most importantly we learned how to drop objects onto a form and make them look like a simple 4-function calculator. Labels, Buttons, TableLayoutPanels, and PictureBoxes all make great sense once you know how to use them. Properties are your friends, you have got to learn what each does and how to use them. How did we add depth to our form? Do you remember which classes and which properties we modified? You need to know this stuff cold.

In our next lesson we will make this calculator do something by writing some event handler code. Should be great fun.